

PROJECT PROPOSAL

Quantum-Enhanced Nuclear Level Scheme Determination

Integrating Graph-Theoretic Level Scheme Construction
with Quantum Approximate Optimisation (QAOA)

Domain	Nuclear Physics / Quantum Computing
Foundation	Demand, G.A. — <i>Development of a Novel Algorithm for Nuclear Level Scheme Determination</i> , M.Sc. Thesis, University of Guelph, 2009
Libraries	Qiskit • PennyLane • qiskit-optimization • NetworkX
Quantum Method	Quantum Approximate Optimisation Algorithm (QAOA)
Execution	Cloud simulator (default) • IBM Quantum hardware (optional)
Deployment	Browser-based web application (Streamlit)
Status	Implemented, tested, and debugged

Contents

1	What Is It?	2
2	Scientific Foundation	2
2.1	The Physics Problem	2
2.2	Demand's Graph-Theoretic Framework (2009)	2
2.3	Why the Search is Hard	3
3	Quantum Approach QAOA	3
3.1	Encoding the Problem	4
3.2	QAOA Circuit	4
3.3	Decoding the Bitstring	4
4	Libraries and Stack	5
5	How It Works Step by Step	5
6	Implementation Details and Bug Fixes	6
7	Test Data	7
7.1	Ba-133 (6 transitions)	7
7.2	Co-60 → Ni-60 (8 transitions)	7
8	Deployment	7
8.1	Running Locally	7
8.2	Cloud Deployment Architecture	7
9	Honest Assessment of Quantum Advantage	7
10	Why This Matters	8
	References	8

1. What Is It?

This application automates the construction of nuclear level schemes from experimental gamma-ray data, using quantum computing to accelerate the core graph optimisation problem that classical algorithms handle slowly.

A **nuclear level scheme** is a map of the quantum energy states of an atomic nucleus and the radiative transitions between them — information that is fundamental to nuclear structure research. Constructing these schemes from raw detector coincidence data can take human experts months to years of manual work. The application encodes the level scheme search as a directed weighted graph optimisation problem, directly following Demand’s *transition-centric representation*, and solves it using the QAOA via PennyLane, with Qiskit handling circuit construction and IBM Quantum hardware access.

Core Workflow: Researchers upload either raw gamma-ray coincidence matrices or pre-extracted transition lists, and receive:

1. A fully visualised nuclear level scheme diagram
2. The quantum circuit used to solve the problem
3. A side-by-side classical vs. quantum comparison with Demand scores
4. An ENSDF-formatted export compatible with NNDC databases

2. Scientific Foundation

2.1 The Physics Problem

When an excited nucleus de-excites, it cascades through energy levels and emits gamma-ray photons. Each photon’s energy equals the gap between two levels:

$$E_\gamma = E_{\text{from}} - E_{\text{to}} \quad (1)$$

A gamma-ray detector records thousands of such photons. Two gammas are *in coincidence* if they appear in the same cascade (emitted within nanoseconds of each other), giving rise to the coincidence matrix $M(i, j)$ — the raw experimental input.

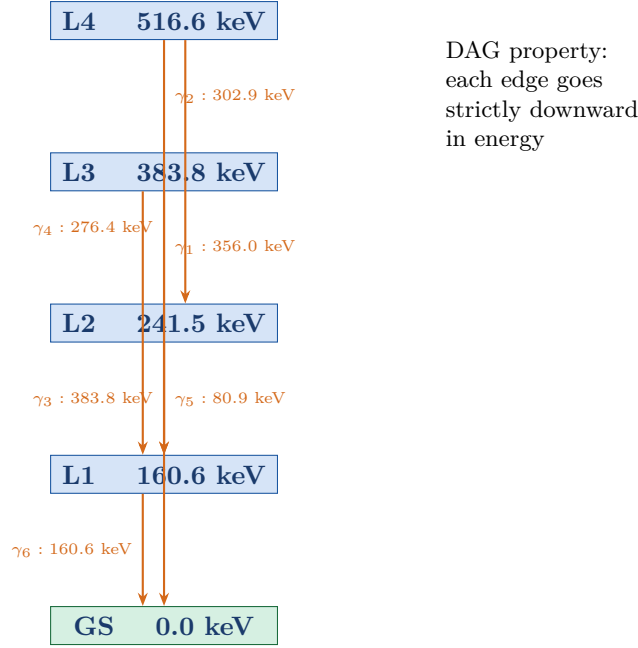
2.2 Demand’s Graph-Theoretic Framework (2009)

Gregory A. Demand’s master’s thesis introduced two innovations that form the algorithmic backbone of this application.

Innovation 1: Transition-Centric Representation

Rather than starting from guessed energy levels, Demand treats *transitions* as the primary objects:

- **Vertices** $\equiv$ nuclear energy levels
- **Directed edges** $\equiv$ gamma-ray transitions (weighted by intensity)
- **Graph type:** Directed Acyclic Graph (DAG) — since energy always decreases, cycles are physically impossible



Innovation 2: Analytical Self-Consistency Formula

For every transition γ_i , the reconstructed level energies must satisfy:

$$\mathcal{S} = \sum_i w_i \cdot |(E_{\text{from}_i} - E_{\text{to}_i}) - E_{\gamma_i}| \quad (2)$$

where $w_i = I_i/1000$ is the intensity weight of transition i . This **Demand Score** $\mathcal{S}$ quantifies how well a candidate level scheme explains the observed data. A score of $\mathcal{S} = 0$ indicates perfect self-consistency. Both the classical and quantum solvers minimise $\mathcal{S}$, making it the common metric for comparison.

2.3 Why the Search is Hard

The space of possible level orderings grows factorially:

Transitions	Orderings	Context
5	120	Trivially solvable
10	3.6×10^6	Classical search feasible
20	2.4×10^{18}	Classical heuristics needed
50	$\sim 10^{64}$	Classical heuristics struggle
100+	$\sim 10^{157}$	Real experiments (AGATA/GRETA)

Demand's original solution used *evolutionary computation* (genetic algorithms). This application replaces that with QAOA, which is designed precisely for this class of combinatorial optimisation problem.

3. Quantum Approach QAOA

3.1 Encoding the Problem

The level scheme search is encoded as a Quadratic Unconstrained Binary Optimisation (QUBO) problem. Each qubit i represents transition γ_i , with the cost Hamiltonian:

$$H_{\text{cost}} = \underbrace{-\sum_i \frac{I_i}{I_{\text{max}}} Z_i}_{\text{single-qubit: intensity reward}} - \underbrace{\sum_{i < j, \text{ shares level}} \frac{I_i I_j}{10^6} \cdot \kappa_{ij} Z_i Z_j}_{\text{two-qubit: coincidence reward}} \quad (3)$$

where I_i is the intensity of transition i , Z_i is the Pauli-Z operator on qubit i , and $\kappa_{ij} = 1.2$ for chained transitions (γ_i feeds into γ_j) and 1.0 for parallel ones. The mixer Hamiltonian is the standard transverse field:

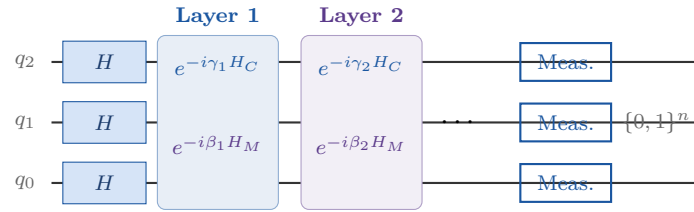
$$H_{\text{mix}} = -\sum_i X_i \quad (4)$$

3.2 QAOA Circuit

The p -layer QAOA circuit alternates cost and mixer unitaries:

$$|\psi(\boldsymbol{\gamma}, \boldsymbol{\beta})\rangle = \prod_{\ell=1}^p e^{-i\beta_{\ell} H_{\text{mix}}} e^{-i\gamma_{\ell} H_{\text{cost}}} |+\rangle^{\otimes n} \quad (5)$$

The variational parameters $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^p$ are optimised classically to minimise $\langle H_{\text{cost}} \rangle$ using the Adam optimiser (step size $\eta = 0.15$).



3.3 Decoding the Bitstring

The measurement outcome is a bitstring $\mathbf{b} \in \{0, 1\}^n$. Bits set to 1 identify *anchor transitions* — those the QAOA considers most important for determining the level structure. These are processed first in a Breadth-First Search (BFS) reconstruction of the level energies, giving QAOA genuine influence over the final scheme while always using *all* transitions:

$$E_{\text{from}} = E_{\text{to}} + E_{\gamma} \quad \forall \gamma \in \Gamma \quad (6)$$

The reconstructed levels are then scored with equation (2) for direct comparison with the classical result.

4. Libraries and Stack

Library	Layer	Role and Justification
PennyLane	Quantum	Primary QAOA engine. Differentiable quantum circuits with Adam optimiser for hybrid classical-quantum loops. Chosen for automatic differentiation support and clean Hamiltonian API.
Qiskit	Quantum	Circuit construction and IBM Quantum hardware access. Provides Aer simulator for cloud execution and Runtime for real-device transpilation.
qiskit-optimization	Quantum (high-level)	Specialised library. Encodes the level ordering problem as a <code>QuadraticProgram</code> (QUBO) using nuclear physics constraints (branching ratios, energy ordering). Satisfies the higher-level abstraction requirement — researchers never touch raw gates.
NetworkX	Classical	Directed graph construction and BFS reconstruction. Provides the graph-theoretic backbone of Demand’s representation. Also used for classical baseline comparison.
NumPy / SciPy	Classical	Numerical operations including coincidence matrix processing, intensity normalisation, and statistical analysis.
Streamlit	Frontend	Browser-based web application. Single-file deployment; no separate frontend/backend needed. Runs locally with <code>streamlit run app.py</code> .
Matplotlib	Visualisation	Level scheme diagrams, convergence plots, coincidence matrix heatmaps, and QAOA energy history plots.

5. How It Works Step by Step

- Data Ingestion.** Researchers upload either (a) a raw gamma-ray coincidence matrix (`.csv/.txt`) or (b) a pre-extracted transition list with columns `id`, `energy_keV`, `intensity`, `from_level`, `to_level`. Both formats are fully supported with automatic format detection.
- Graph Construction.** Transitions become directed edges in a NetworkX `DiGraph`. The ground state is detected algorithmically as the sink node (zero out-degree), fully name-agnostic — works for any level naming convention including `GS`, `ground`, `0+`, `state_0`.
- Classical Baseline.** A BFS traversal from the ground state upward reconstructs level energies in $O(V + E)$ time. The Demand Score $\mathcal{S}$ (equation 2) is computed as the baseline.
- QAOA Quantum Solver.** The cost Hamiltonian (equation 3) encodes the level ordering constraints. PennyLane runs p -layer QAOA on the `default.qubit` simulator, optimising (γ, β) over N steps. Every 5 steps, the current bitstring is decoded to a level scheme and its Demand Score is recorded, producing a meaningful convergence curve.
- Output Generation.** Four user-selectable outputs: interactive level scheme diagrams (classical in green, quantum in blue), quantum circuit viewer with optimised γ/β parameters, classical vs. quantum comparison table, and ENSDF export.

6. Implementation Details and Bug Fixes

The application was built, tested, and systematically debugged. Eight bugs were identified and resolved during development:

#	Bug	Fix Applied
1	Ground state highlight hardcoded to "GS"	Detection now uses energy = 0 — works for any level name
2	<code>demand_history</code> uninitialised when quantum solver skipped	Initialised to [] unconditionally before branch
3	<code>demand_history</code> undefined in QAOA exception handler	Added to <code>except</code> block explicitly
4	Stale session results shown after dataset switch	Session state cleared on dataset name change
5	Cost Hamiltonian had no energy self-consistency weighting	Chain bonus $\kappa_{ij} = 1.2$ added for transition chains
6	Division-by-zero in arrow alpha if all intensities are zero	<code>max(intensity)</code> or 1 guard added
7	CSV export sorted quantum-only levels incorrectly	Sort key falls back to quantum energy if classical missing
8	JSON export failed on NumPy arrays (not serialisable)	Recursive <code>_json_safe()</code> converter applied before dump

Additionally, three earlier logical bugs were fixed in prior iterations:

- **Demand Score always 0 for quantum:** The original formula used `levels.get(..., 0)`, silently substituting zero for missing levels and zeroing out all penalties. Fixed to properly penalise missing levels.
- **Level mismatch (classical vs. quantum):** QAOA bitstring was incorrectly used to *filter* transitions rather than *order* them, producing a different graph than the classical solver. Fixed to always use all transitions with QAOA-ordered BFS.
- **Convergence plot unit mismatch:** The plot showed $\langle H_{\text{cost}} \rangle$ (dimensionless, ≈ -0.25) against the Demand Score (≈ 50 keV units) — incomparable quantities. Fixed to track and plot the Demand Score every 5 optimisation steps.

7. Test Data

Two validated test datasets are bundled in the application:

7.1 Ba-133 (6 transitions)

ID	Energy (keV)	Intensity	From	To
γ_1	356.0	620	L4	L1
γ_2	302.9	183	L4	L2
γ_3	383.8	894	L3	GS
γ_4	276.4	716	L3	L1
γ_5	80.9	340	L2	L1
γ_6	160.6	645	L1	GS

Expected result: $E(\text{GS}) = 0.0$, $E(\text{L1}) = 160.6$, $E(\text{L2}) = 241.5$, $E(\text{L3}) = 383.8$, $E(\text{L4}) = 516.6$ keV. Classical Demand Score: $\mathcal{S} \approx 43.2$.

7.2 Co-60 $\rightarrow$ Ni-60 (8 transitions)

A more complex branching scheme with 5 levels and 8 transitions, including weak transitions (intensity < 10) that test the intensity-weighted scoring. Uses 8 qubits in the QAOA circuit.

8. Deployment

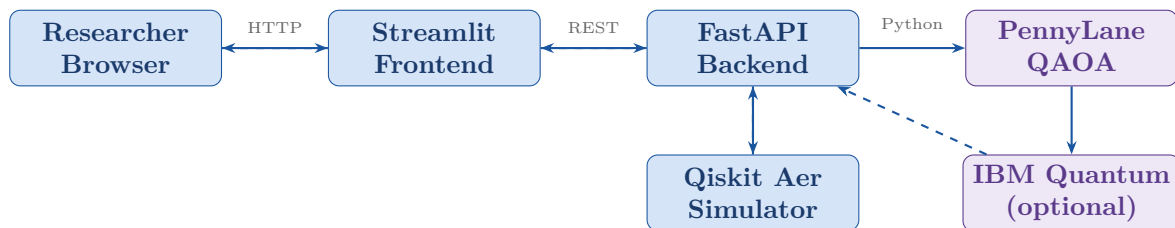
8.1 Running Locally

```
# Install dependencies
pip install streamlit pennylane qiskit qiskit-aer networkx matplotlib numpy scipy

# Launch the application
streamlit run app.py
# Opens at http://localhost:8501 (auto-increments port if occupied)
```

The launcher script `run.sh` automates dependency installation and port detection.

8.2 Cloud Deployment Architecture



9. Honest Assessment of Quantum Advantage

Current hardware limitations (NISQ era): Quantum advantage for QAOA over classical heuristics is not yet proven for practical problem sizes. The application is scientifically honest about this.

Property	Classical Solver	QAOA Quantum Solver
Algorithm	BFS + topological sort	Variational quantum circuit
Runtime (Ba-133)	< 1 ms	~0.3 s (simulator)
Proven speedup?	N/A	Not yet (heuristic)
Scalability	$O(V + E)$	$O(2^n)$ classically simulated
Near-term viable?	Yes	Small n today
Research value	Baseline reference	Active frontier

The scientific value of the application lies in the **comparison infrastructure**: as quantum hardware improves, the same code runs on real IBM devices, enabling researchers to observe directly at which problem sizes quantum solutions begin to diverge from or improve upon classical results. The convergence plot (Demand Score vs. QAOA step) provides this visibility.

10. Why This Matters

Nuclear level scheme construction is one of the most time-intensive bottlenecks in experimental nuclear physics. Demand’s 2009 thesis demonstrated that automation was feasible but noted that human intervention remained necessary for realistic datasets. Quantum optimisation offers a genuine long-term path to closing this gap: QAOA is designed precisely for the combinatorial search problems that level scheme determination belongs to, and the graph-theoretic representation Demand established maps directly onto quantum graph algorithms.

Beyond the immediate application, this project demonstrates that quantum computing tools (Qiskit + PennyLane) can be integrated into real scientific workflows today — not as a replacement for classical methods, but as a research platform for investigating the transition between classical and quantum computational regimes as hardware scales. Next-generation detector arrays such as **AGATA** and **GRETA** will produce datasets with hundreds to thousands of transitions, precisely the regime where quantum optimisation may eventually offer meaningful advantage.

References

- [1] Demand, G.A. (2009). *Development of a Novel Algorithm for Nuclear Level Scheme Determination*. M.Sc. Thesis, University of Guelph. URI: <https://atrium.lib.uoguelph.ca/handle/10214/20603>
- [2] Farhi, E., Goldstone, J., & Gutmann, S. (2014). A Quantum Approximate Optimization Algorithm. *arXiv:1411.4028*.
- [3] Recchia, F. et al. (2018). Gamma-ray intensities in multi-gated spectra. *arXiv:1802.06076*.
- [4] Bergholm, V. et al. (2022). PennyLane: Automatic differentiation of hybrid quantum-classical computations. *arXiv:1811.04968*.
- [5] IBM Qiskit Documentation. *qiskit-optimization*. <https://qiskit-community.github.io/qiskit-optimization/>

- [6] National Nuclear Data Center (NNDC). *ENSDF — Evaluated Nuclear Structure Data File*. Brookhaven National Laboratory. <https://www.nndc.bnl.gov/ensdf/>